

VPN Log Investigation with ELK: Brute-Force Attack and Account Compromise

TryHackMe — Investigating with ELK 101 (SOC Level 1 path) | July 2026

Goal / Objective

This lab is built around a scenario at a fictional company called CyberT. The SOC team received complaints about failed VPN connections, and the job was to dig through a month of VPN logs in Kibana and work out what actually happened. The guided part of the room teaches the core ELK skills: exploring logs in Discover, writing KQL queries, building visualizations in Lens, and assembling a monitoring dashboard.

After finishing the guided tasks, I kept investigating on my own. The room told me there were 274 failed logins. It did not tell me when they happened, how fast they came in, or whether the attacker ever got in. Those are the questions a SOC analyst would actually be asked, so I went and answered them.

Tools Used

- Elastic Stack (Elasticsearch + Kibana) — Discover, Lens, and Dashboards
- KQL (Kibana Query Language) — free text, wildcards, field:value queries, and/or/not logic
- TryHackMe SOC Level 1 lab environment (browser-based Kibana instance)
- Dataset: vpn_connections index — 2,857 VPN log events from January 2022
- Microsoft Word — case study documentation

What I Did

Step 1 — Connected to the lab and opened Kibana

I deployed the lab machine from the TryHackMe room and signed into the Kibana instance in my browser. From the welcome screen I headed straight for the Analytics tools, since Discover is where log analysis starts.

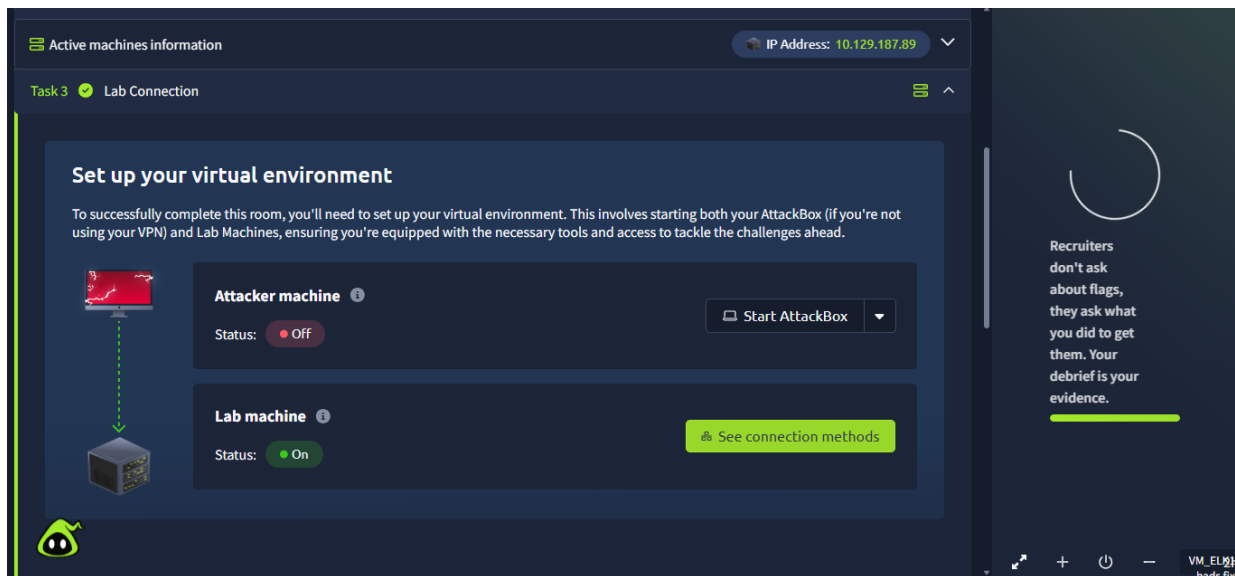


Figure 1. Setting up the lab environment in the Investigating with ELK 101 room.

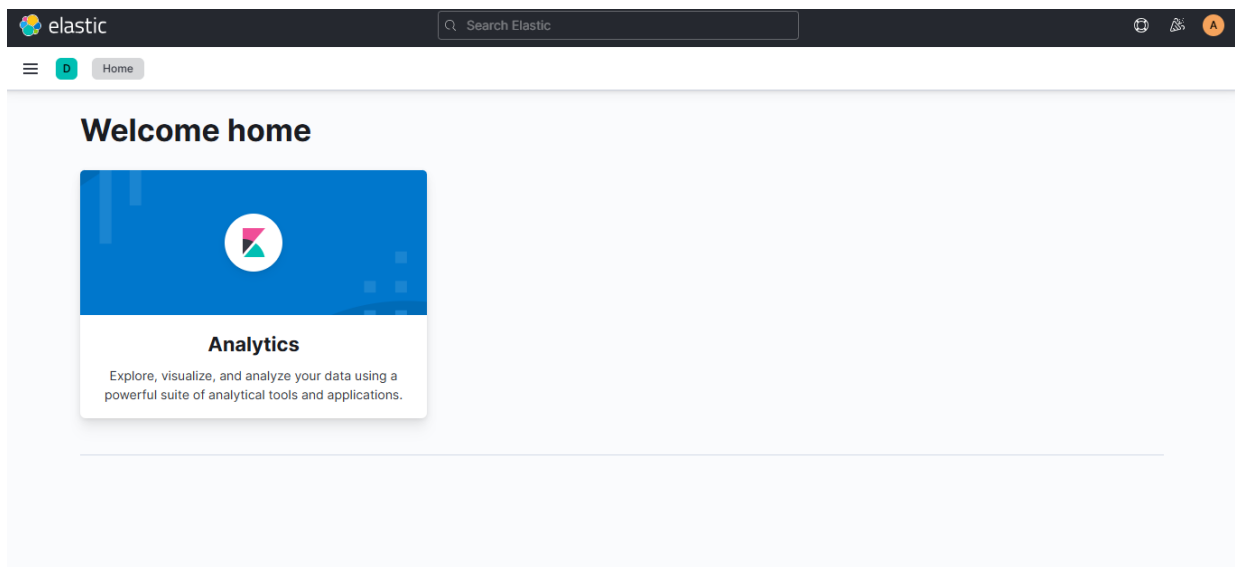


Figure 2. Kibana welcome screen after signing into the lab instance.

Step 2 — Loaded the VPN logs and set the right time window

I selected the `vpn_connections` index and immediately hit the first lesson of the room: the default Last 15 minutes window returned nothing, because the logs are from January 2022. Widening the time range to cover the dataset brought back 2,861 events; the room's January-focused window works out to 2,857 events, which the record counts later in the lab confirm.

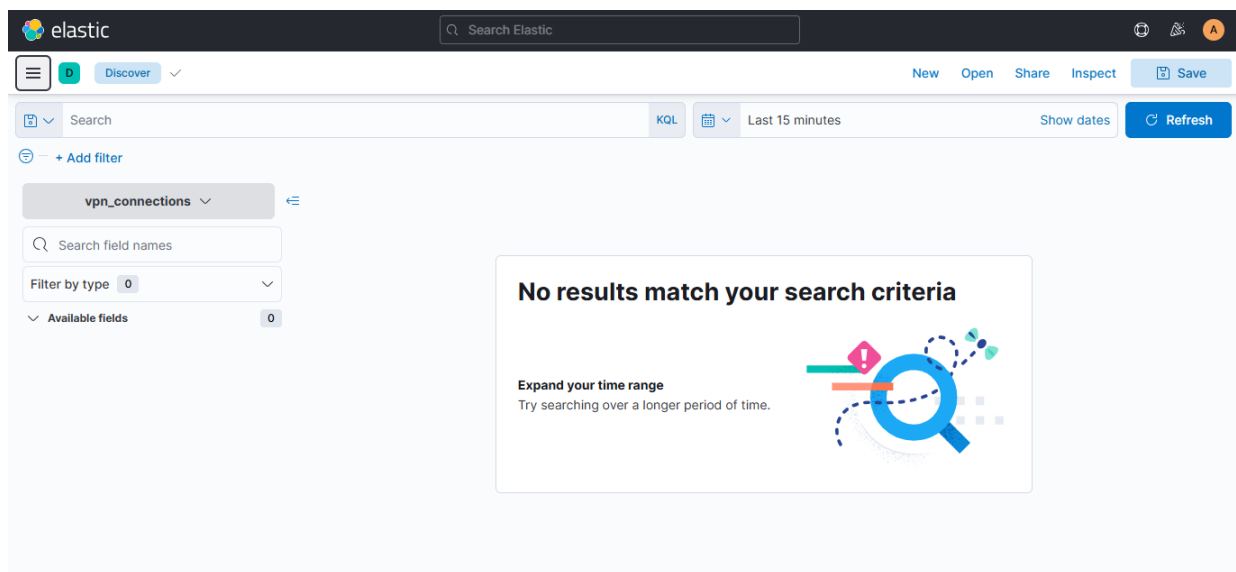


Figure 3. The default 15-minute window returns no results — the data is from January 2022.

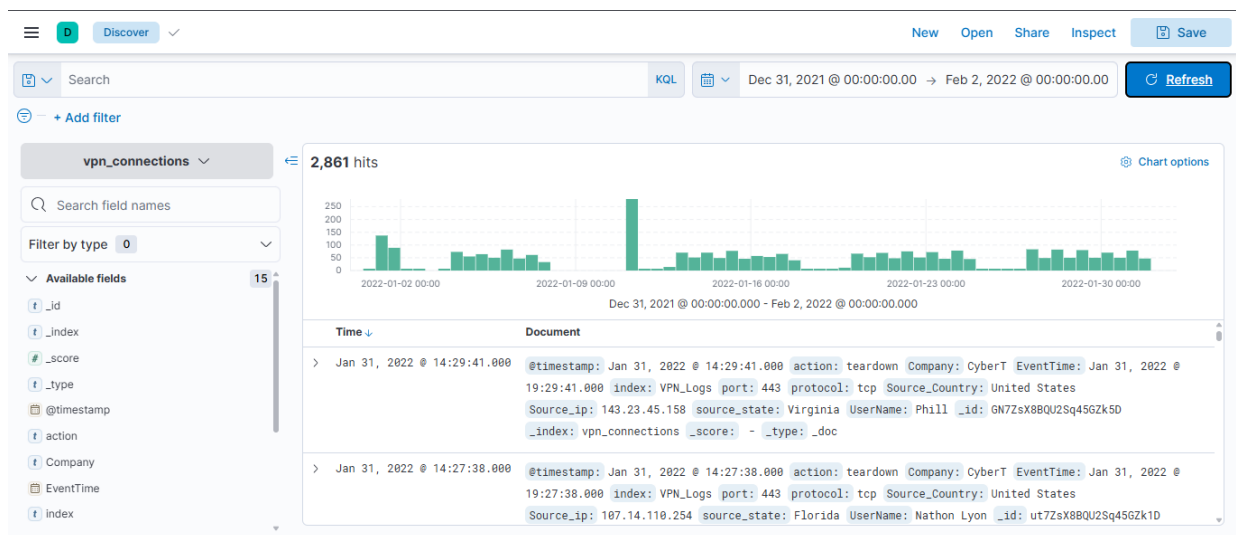


Figure 4. Time range widened to cover the dataset: 2,861 events with the event histogram.

Step 3 — Profiled the data before writing any queries

Clicking through the field list gave quick answers with zero query writing. The busiest source IP in the dataset was 238[.]163[.]231[.]224, and James was the most active user. To make the raw logs easier to scan, I trimmed the document table down to just Source_ip, Username and Source_Country, and saved that view as vpn-connections-table so I could pull it back up any time.

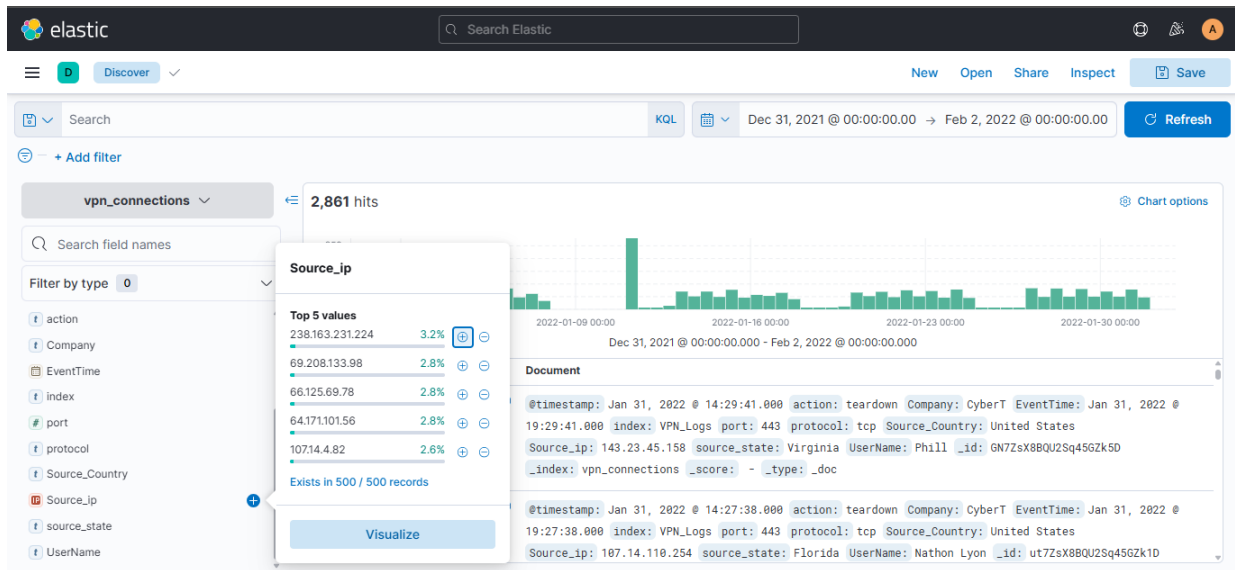


Figure 5. Source_ip quick stats — 238.[.]163.[.]231.[.]224 is the most active source IP.

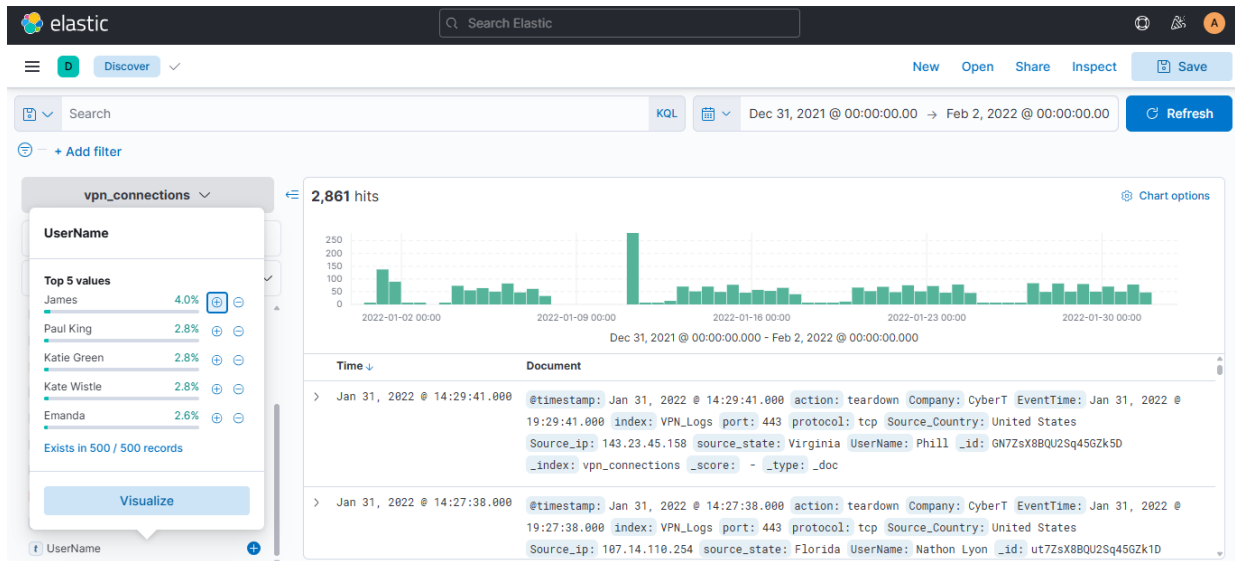


Figure 6. UserName quick stats — James generated the most VPN events.

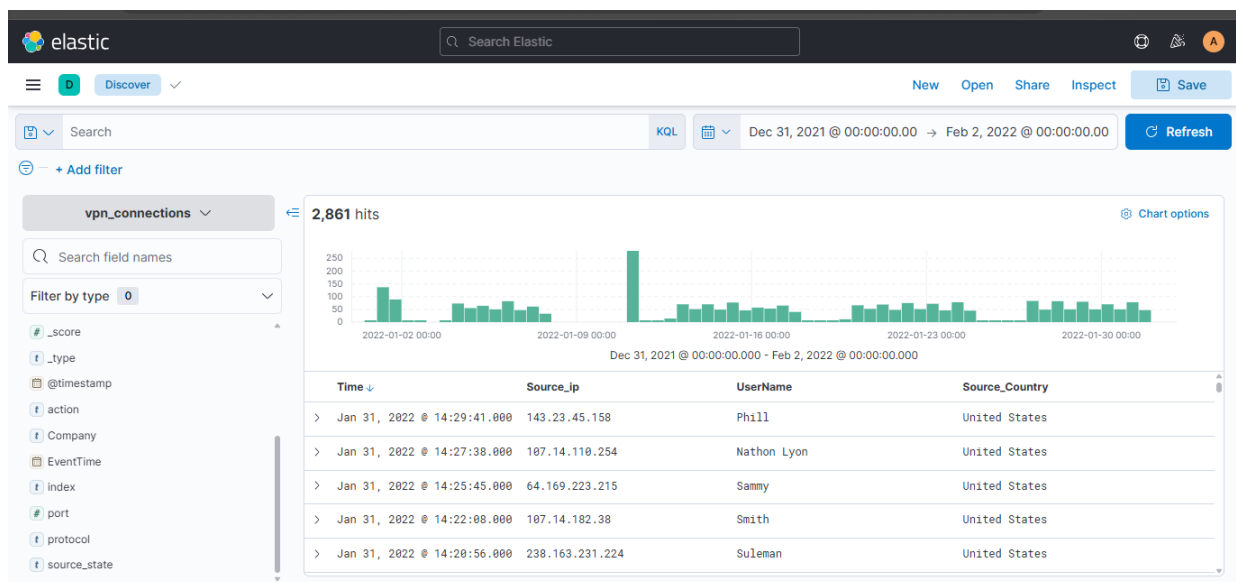


Figure 7. Discover table trimmed to the three columns that matter for triage.

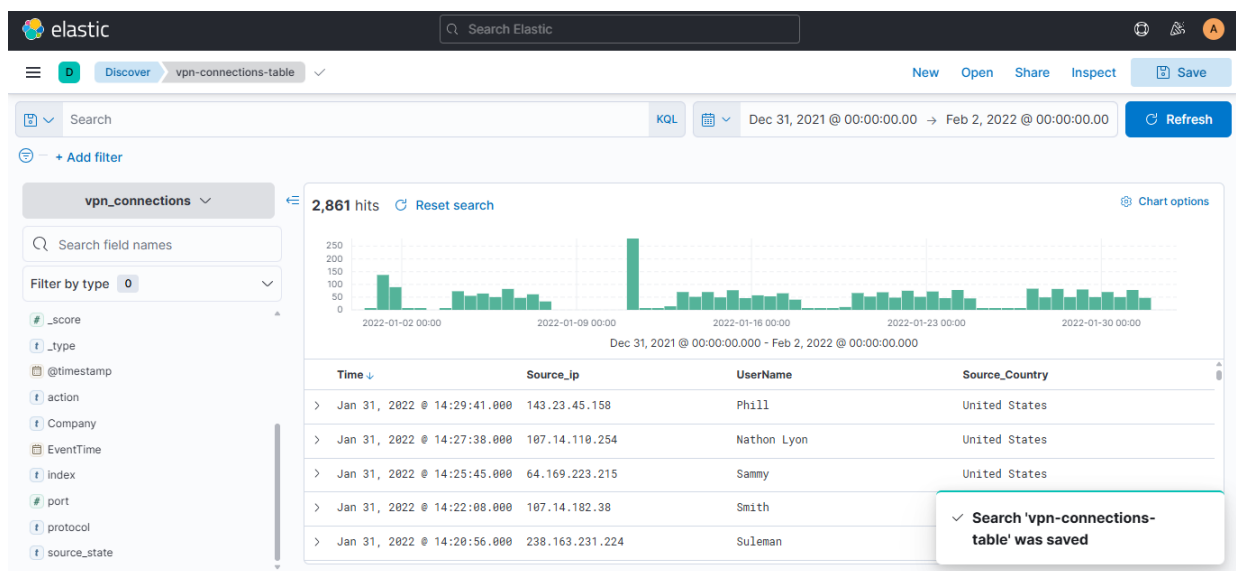


Figure 8. The reusable view saved as vpn-connections-table.

Step 4 — Answered targeted questions with filters and KQL

With the layout sorted, I moved on to the actual questions. Filtering on Emanda returned her 56 connections, and the field stats showed her top source IP was 107[.]14[.]1[.]247. The event histogram had one obvious spike on January 11, so I drilled into that morning: 280 events, and 98.2 percent of them came from a single IP — 172[.]201[.]60[.]191. That was the first appearance of the address that would define the rest of this investigation. I then combined a filter with an exclusion to count connections from the high-volume IP outside New York (48 events), and ran the check that mattered most: Johnny Brown was terminated on January 1, so his account should have gone quiet. It did not. His account built one successful VPN connection on January 7 at 03:28:47 from 175[.]20[.]60[.]191, six days after he was let go. Every room answer validated correct.

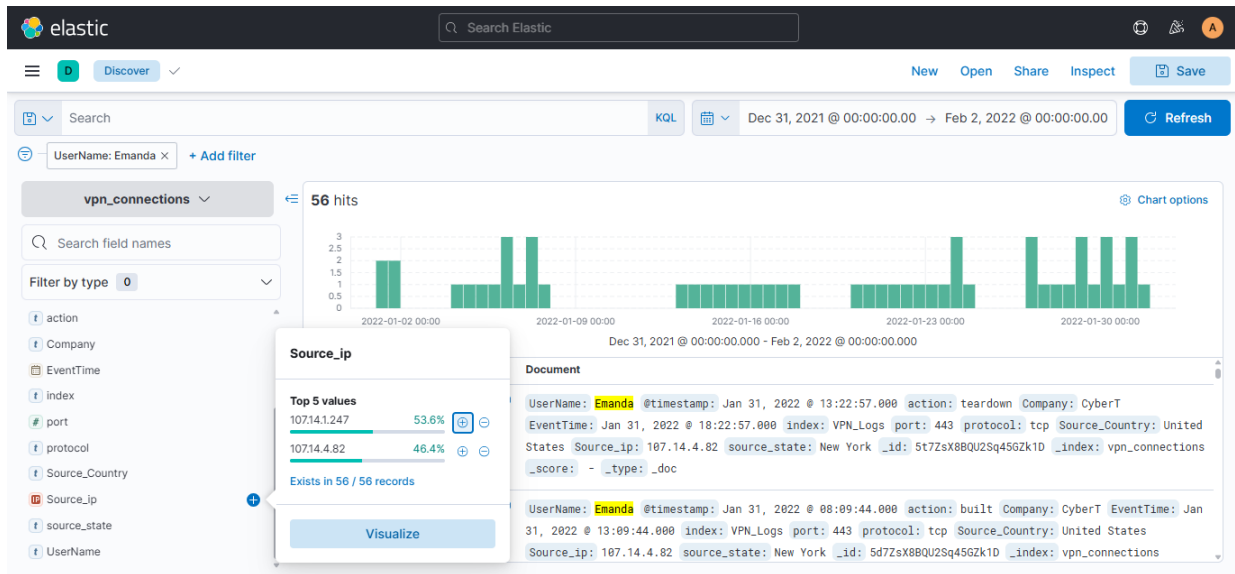


Figure 9. Emanda's 56 connections, with her top source IP 107.[.14.].1[.].247 in the field stats.

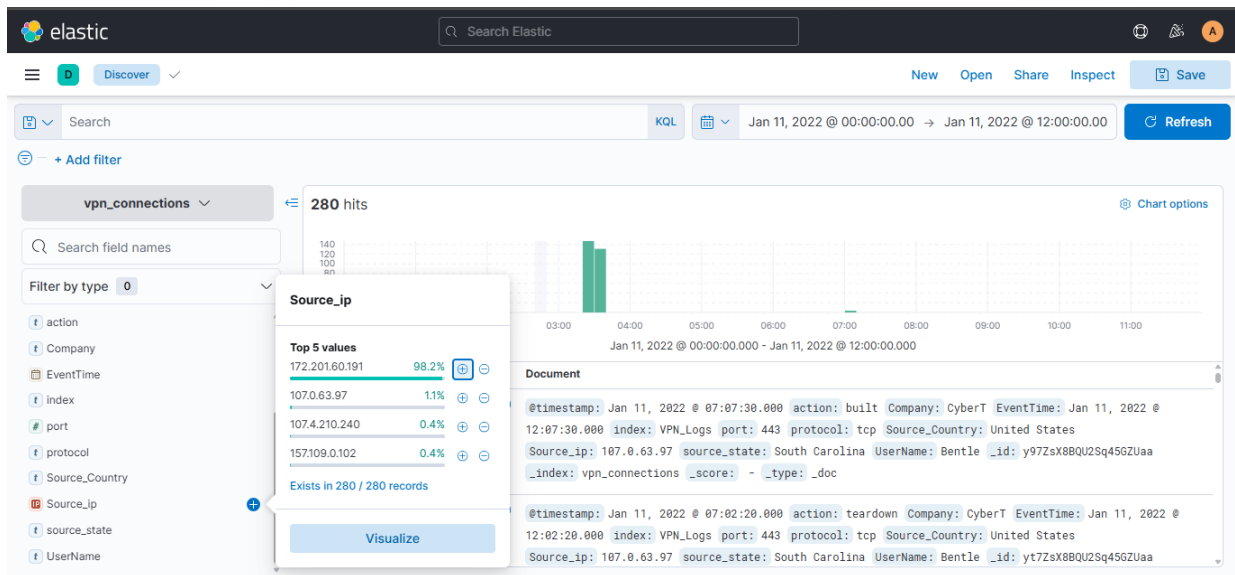


Figure 10. Drilling into the January 11 spike: 280 events that morning, 98.2% from 172.[.201.].60[.].191.

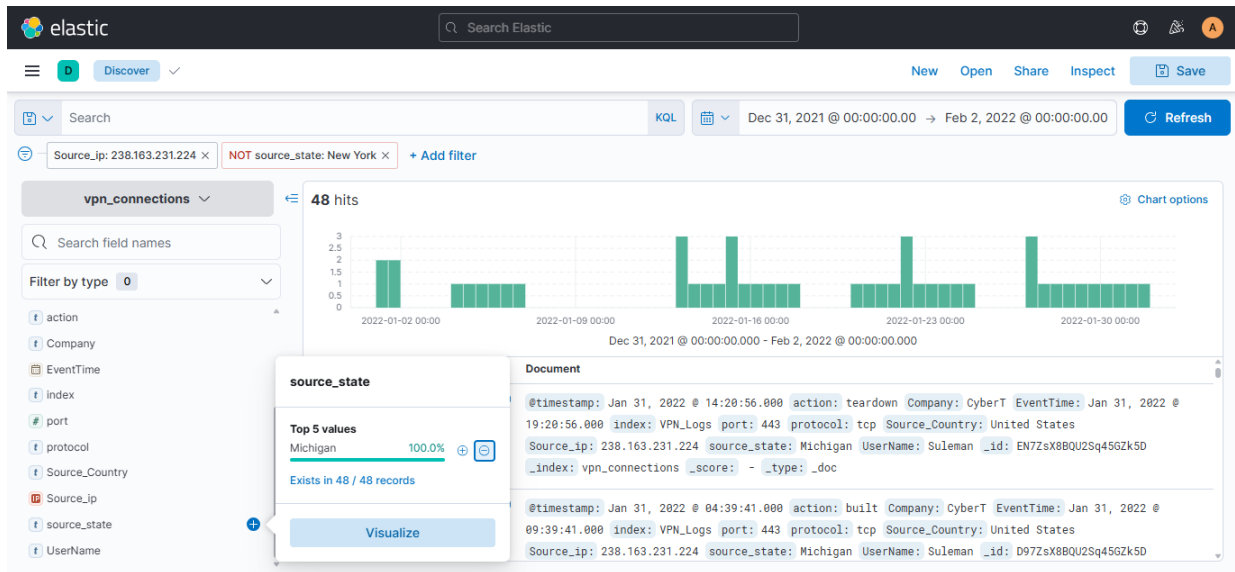


Figure 11. Filter plus exclusion: 48 connections from 238[.]163[.]231[.]224 outside New York.

The screenshot shows the TryHackMe interface for the Discover task. The questions and answers are as follows:

- Question: On 11th Jan, which IP caused the spike observed in the time chart?
Answer: 172.201.60.191 (Correct Answer)
- Question: How many connections were observed from IP 238.163.231.224, excluding the New York state?
Answer: 48 (Correct Answer)
- Question: Create a table with the fields IP, UserName, Source_Country and save.
Answer: No answer needed (Correct Answer)

The interface also shows the IP Address: 10.129.187.89 and a KQL Overview section at the bottom.

Figure 12. The Discover task questions all confirmed correct in TryHackMe.

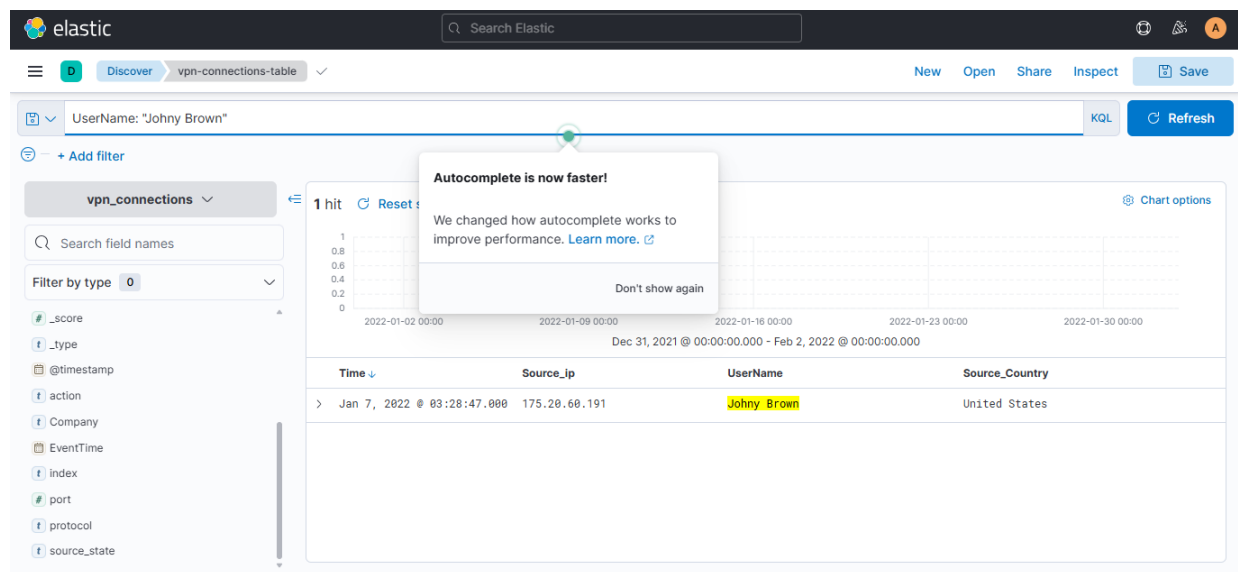


Figure 13. Terminated-employee check: one successful connection on Johny Brown's account, Jan 7 at 03:28:47.

The screenshot shows the KQL task questions interface. It contains two questions about creating search queries to filter logs. The first question asks for the number of records returned for Source_Country 'United States' and User 'James or Albert', with the answer '161' confirmed as correct. The second question asks for the number of VPN connections observed after the termination of User 'Johny Brown' on Jan 1, 2022, with the answer '1' confirmed as correct.

Task 6 Creating Visualizations

Task 7 Creating Dashboards

Figure 14. The KQL task questions confirmed correct — including the post-termination connection count.

Step 5 — Built two Lens visualizations

I built a table of traffic by country and saved it to the library as Country with TOP traffic (United States on top with 2,304 events, then Canada, England, Israel and Singapore — the finished table is visible on the dashboard in Step 6). The second visualization, Failed VPN Attempts by User, taught me two things. First, my Lens workspace came up empty until the tip in the corner pointed out the problem: the time range did not cover the data. Small thing, easy to lose twenty minutes on. Second, once I applied the failed-connections filter, the UserName field collapsed to a single value: Simon, 100 percent of all 274 failed documents. One user, one IP, every failure in the month.

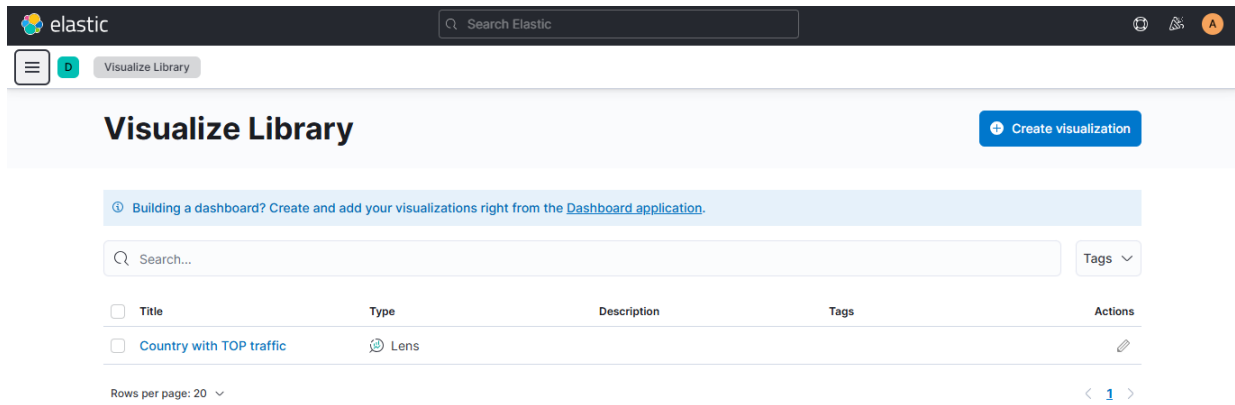


Figure 15. Country with TOP traffic saved to the Visualize Library.

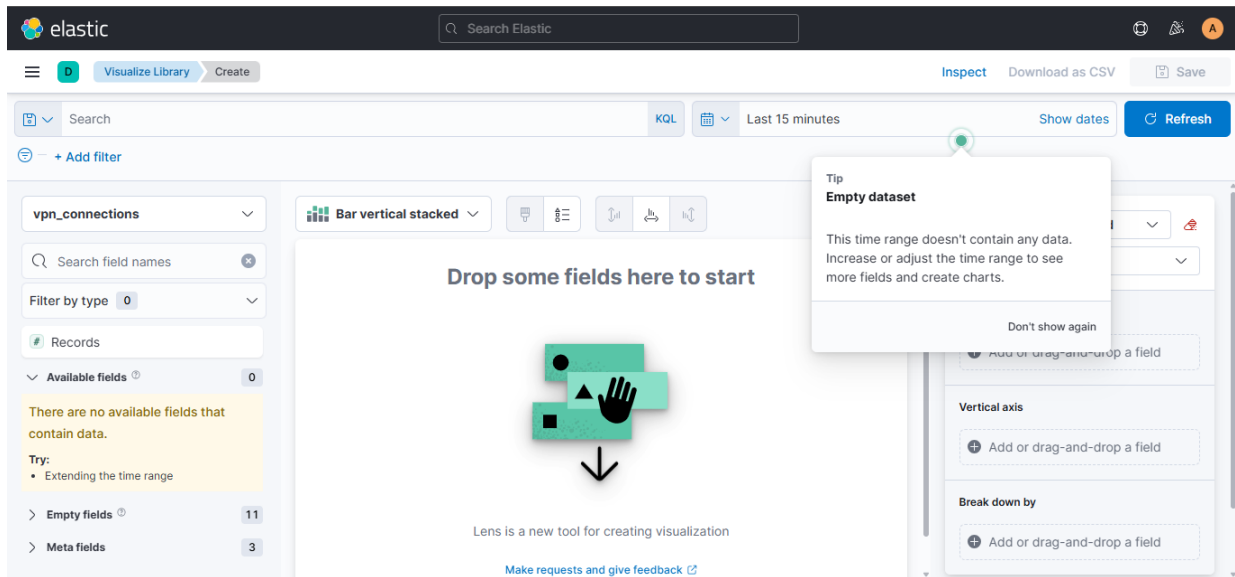


Figure 16. The empty-dataset tip — the Lens time range didn't cover the data. Fixed and moved on.

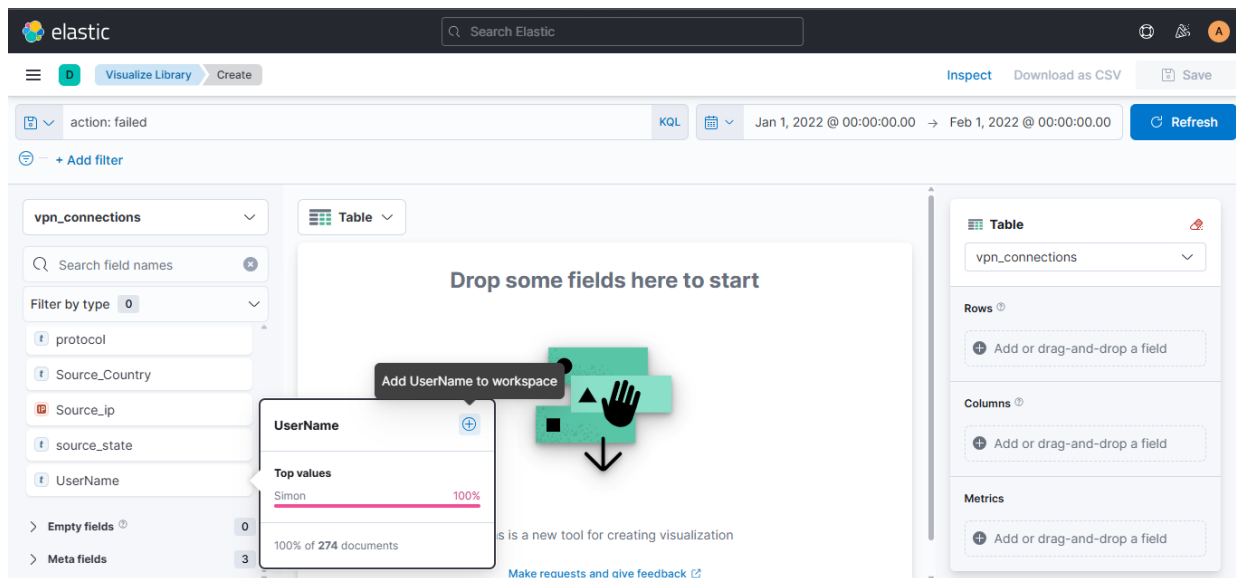


Figure 17. Building Failed VPN Attempts by User: with the failed filter on, Simon is 100% of all 274 documents.

Step 6 — Assembled and saved the VPN Monitoring Dashboard

I created a new dashboard, added both saved visualizations from the library, set the January time range, and saved it as VPN Monitoring Dashboard. TryHackMe confirmed the dashboard task complete, which closed out the guided part of the room.

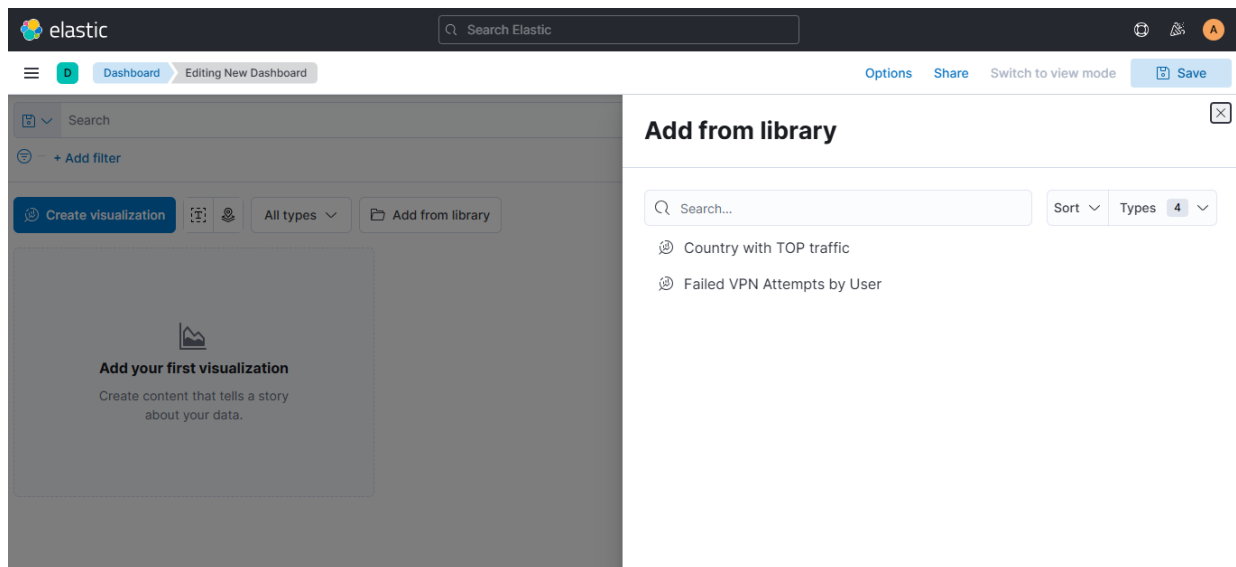


Figure 18. Adding both saved visualizations to the dashboard from the library.

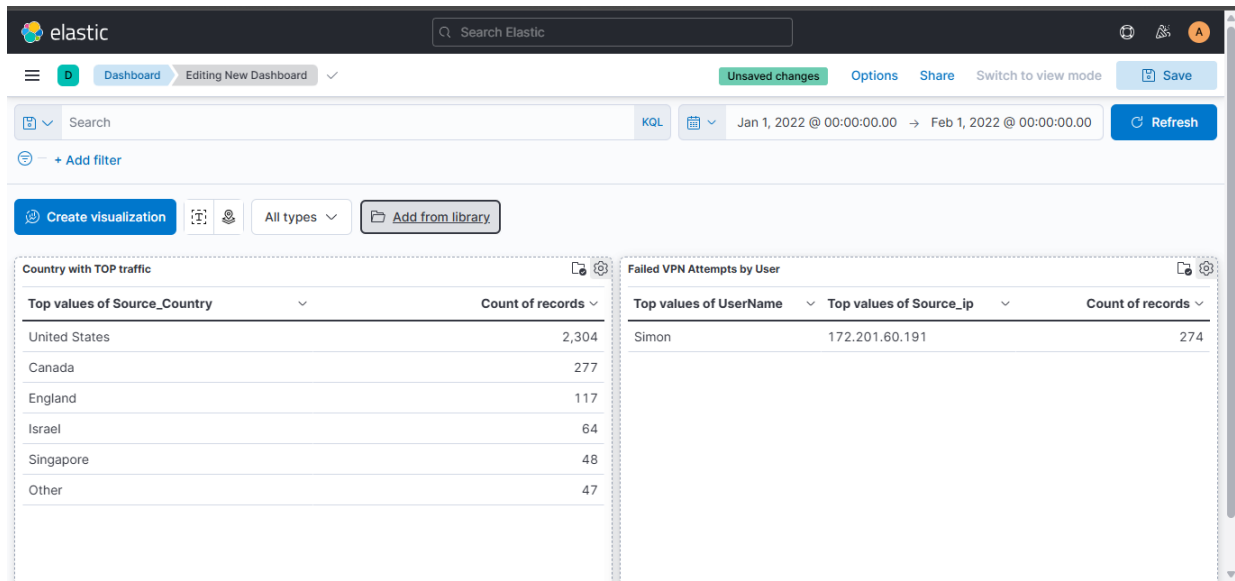


Figure 19. Both panels placed with the January 2022 time range applied.

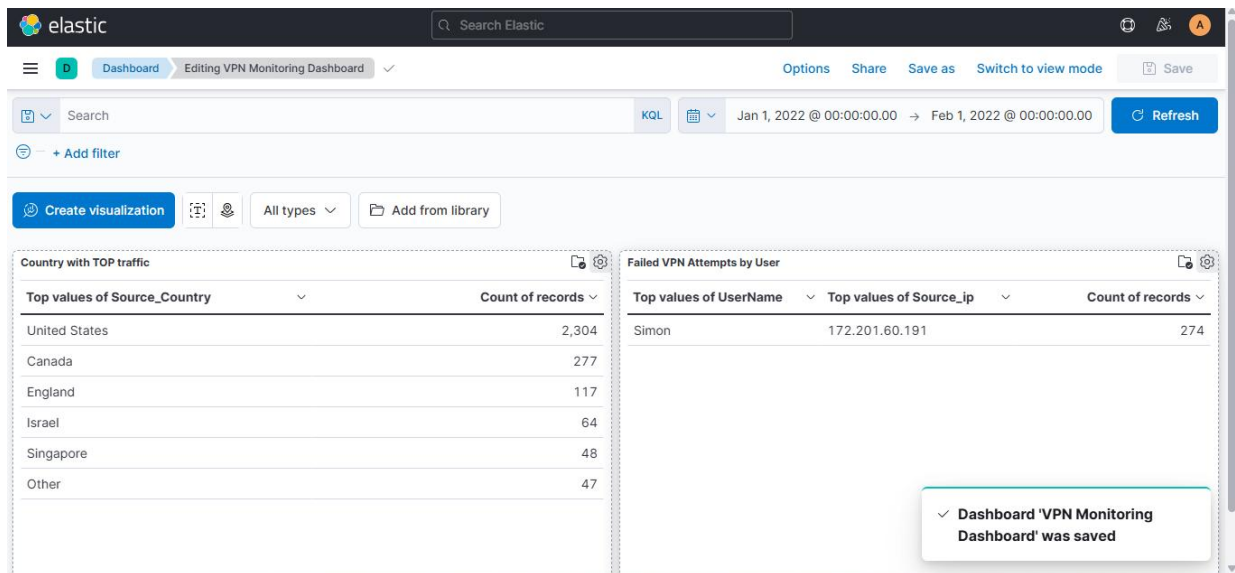


Figure 20. Dashboard saved as VPN Monitoring Dashboard.

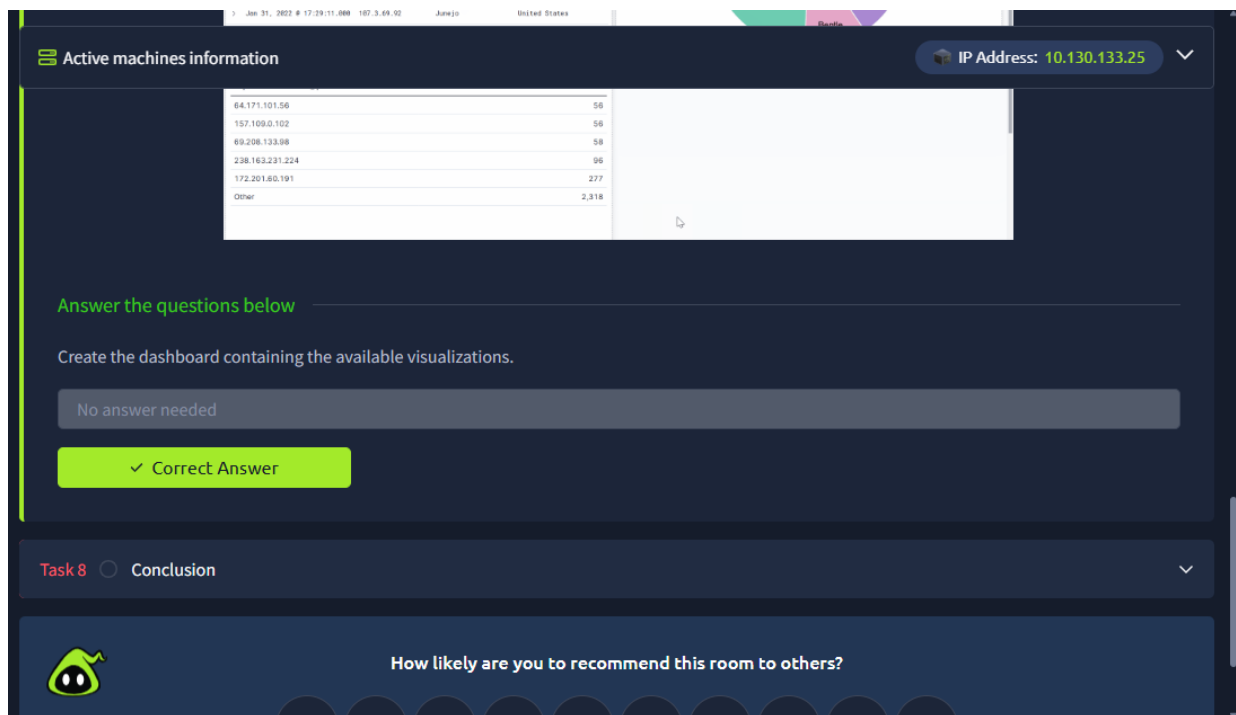


Figure 21. TryHackMe marking the dashboard task complete, with the room conclusion below.

Step 7 — Went beyond the room: isolated the failures and put them on a timeline

The number 274 kept bothering me. A count is a lead, not a finding. My first attempt to filter the dashboard used placeholder syntax that broke both panels — but the KQL autocomplete turned that mistake into the answer, showing me the action field's real values: built, failed and teardown. Filtering on action:"failed" made both panels agree perfectly: 274 failures, one user, one IP, one country.

Then I built a bar chart of failures over time. The first version quietly charted all 2,857 events because the filter had not carried into Lens — I caught it from the record count in the suggestions panel and re-applied the filter until it read 274. Zooming into January 10–12 at per-hour resolution showed the real picture: every single failure landed inside one overnight hour on January 11, roughly four to five attempts per minute. No human types that fast for an hour straight. This was an automated brute-force attack.

I added the chart to the dashboard, titled it Failed VPN Attempts Over Time, and attached action: failed as a panel-level filter — that way the chart stays accurate even when the dashboard's own search bar is cleared and the other two panels show all traffic. I cleared the search bar, verified all three panels, and saved the dashboard.

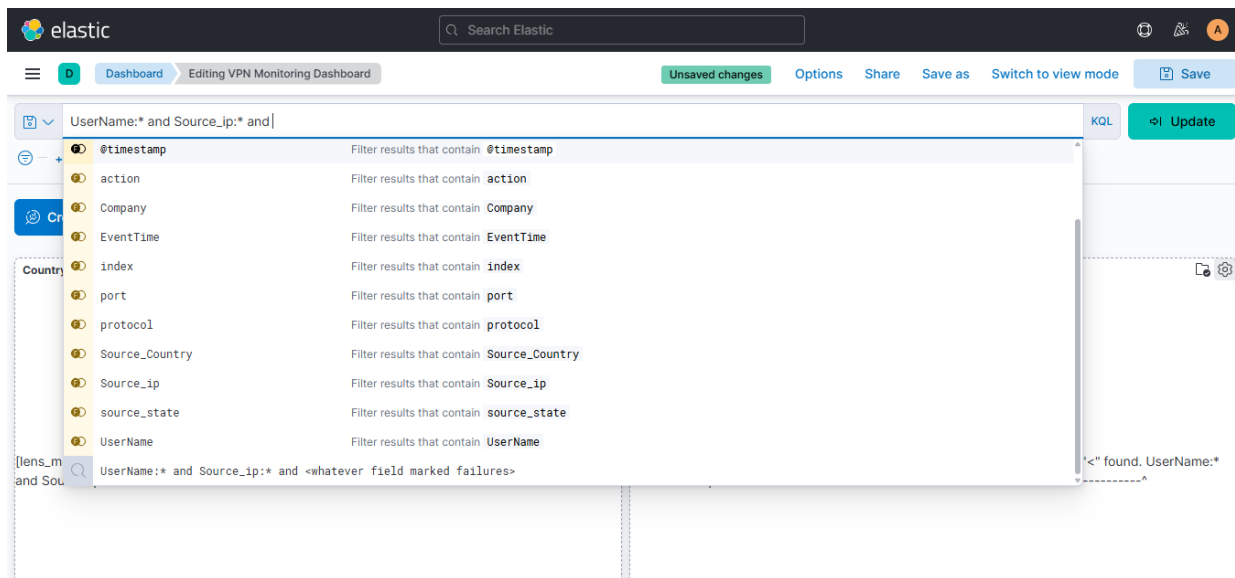


Figure 22. First filter attempt with placeholder syntax — it failed, and that was useful.

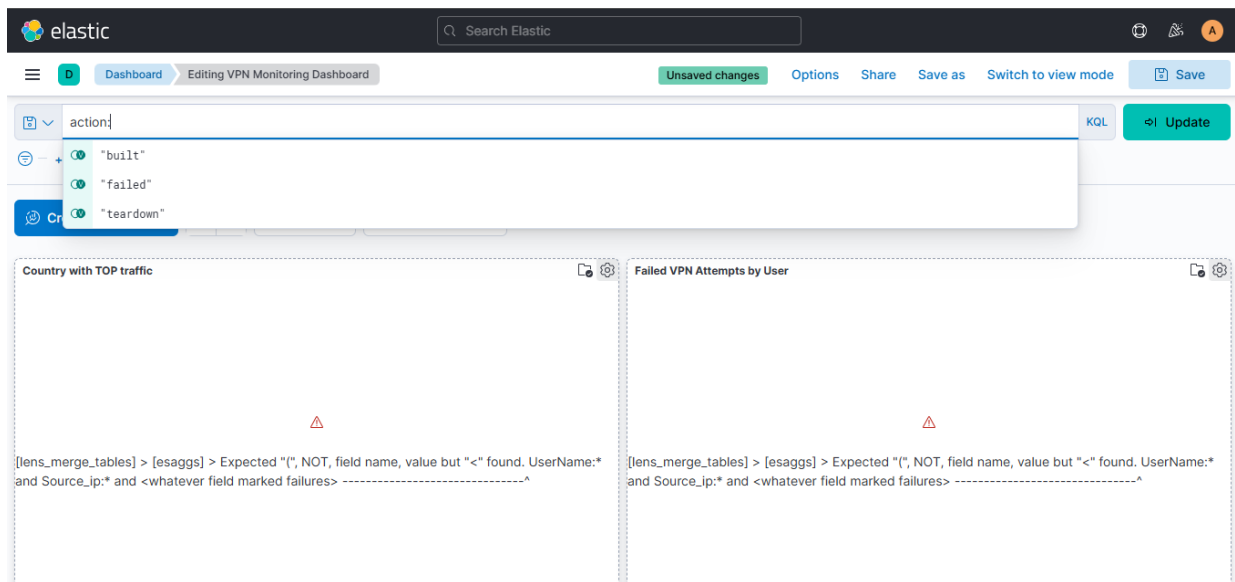


Figure 23. KQL autocomplete revealing the action field's values: built, failed, teardown.

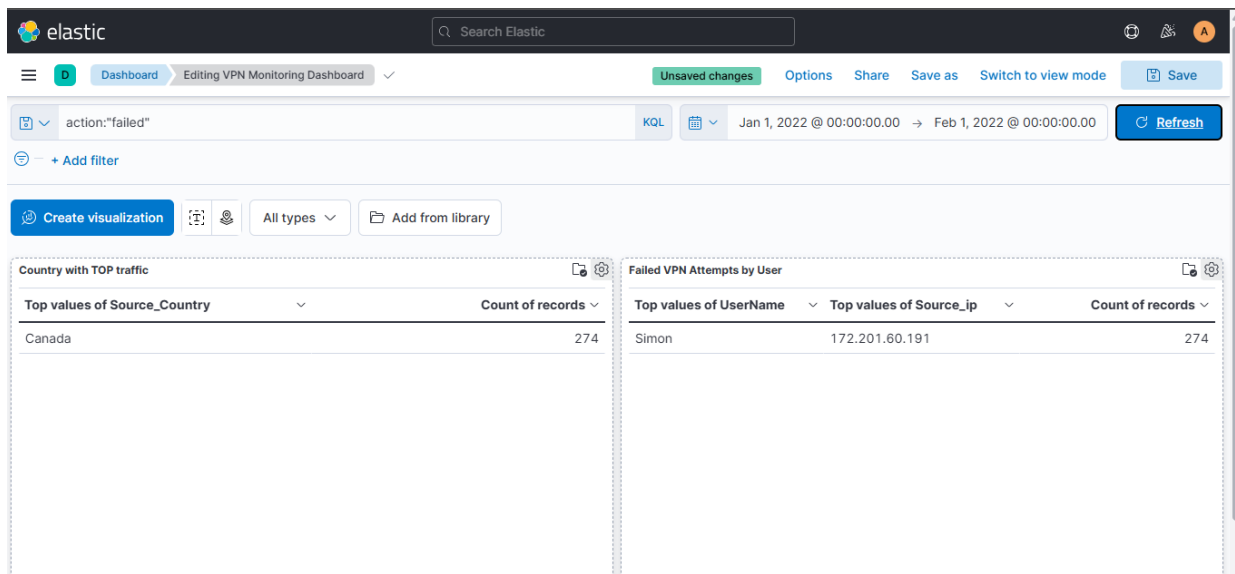


Figure 24. Dashboard filtered on action:"failed" — both panels agree: 274 failures, all Simon, all from 172[.]201[.]60[.]191.

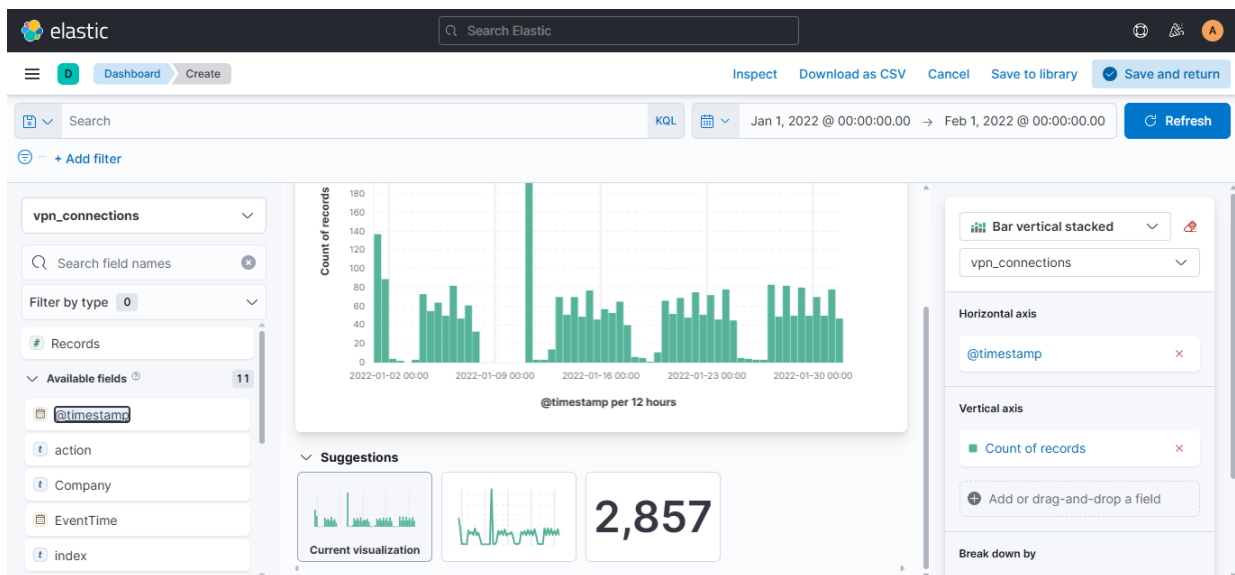


Figure 25. The catch: the unfiltered chart was counting all 2,857 events, not just failures.

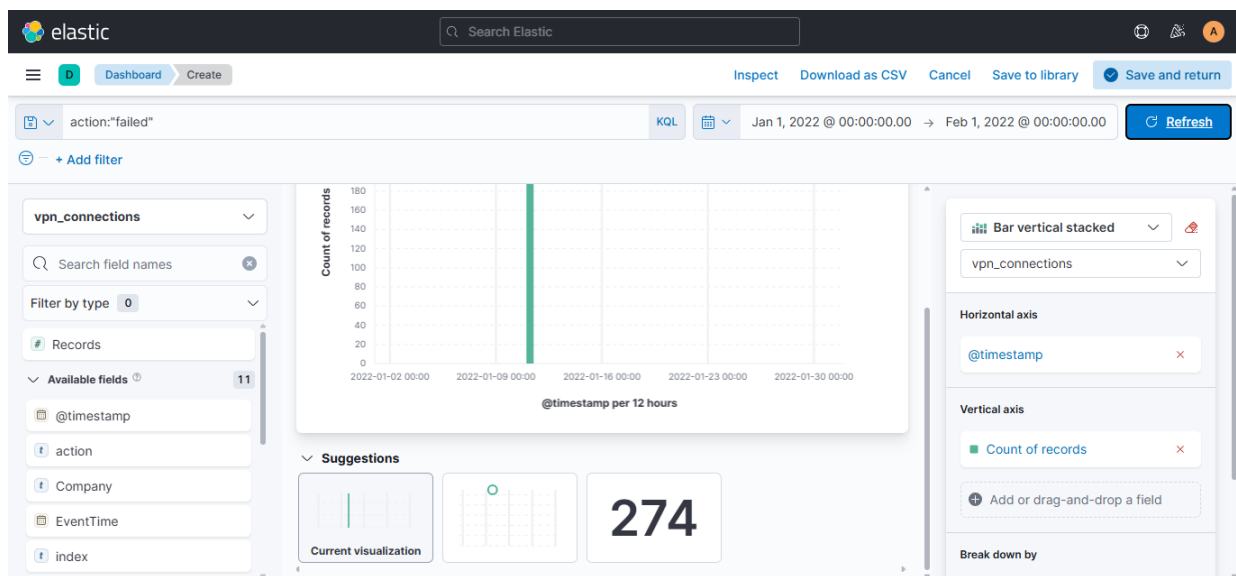


Figure 26. Filter re-applied — the record count now reads 274.

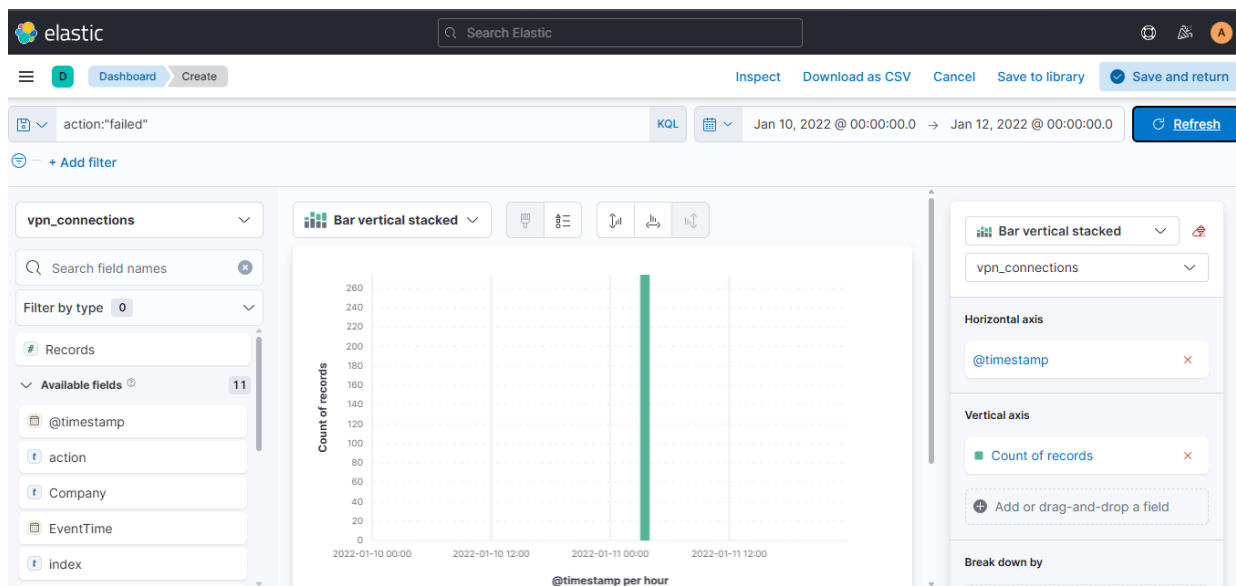


Figure 27. Zoomed to per-hour resolution: every failure sits inside a single overnight hour on January 11.

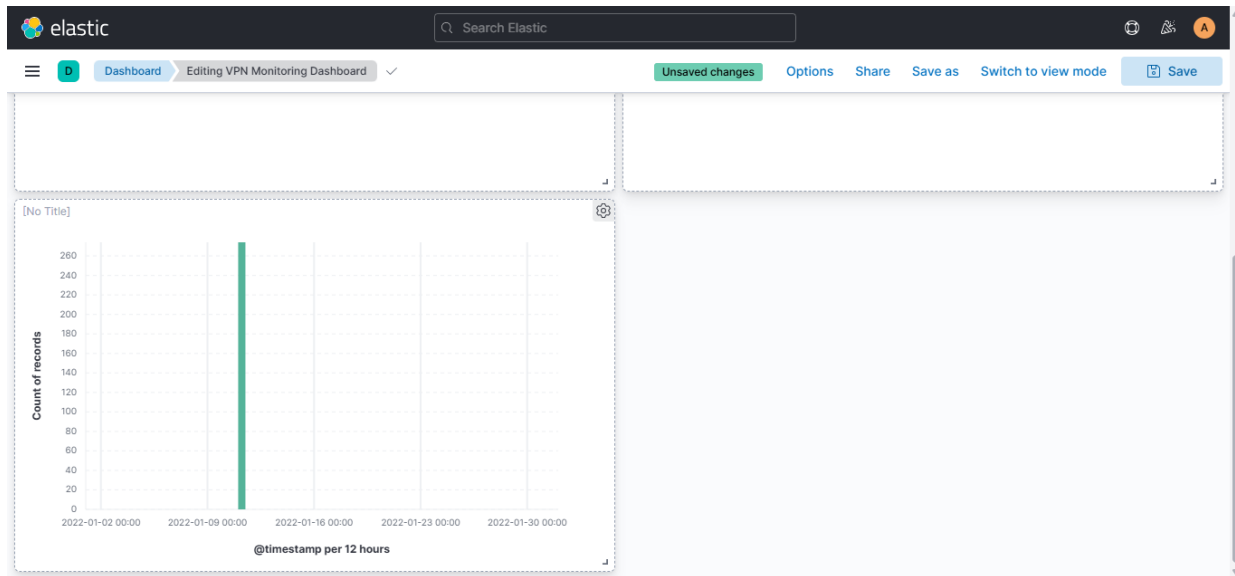


Figure 28. The new timeline panel added to the dashboard.

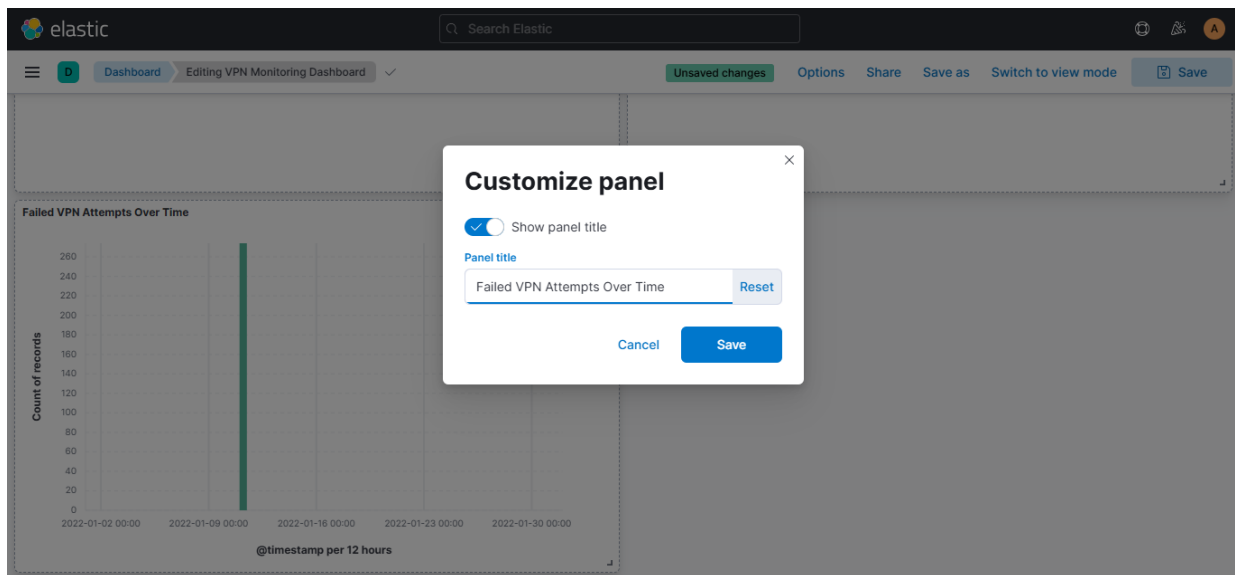


Figure 29. Naming the panel: Failed VPN Attempts Over Time.

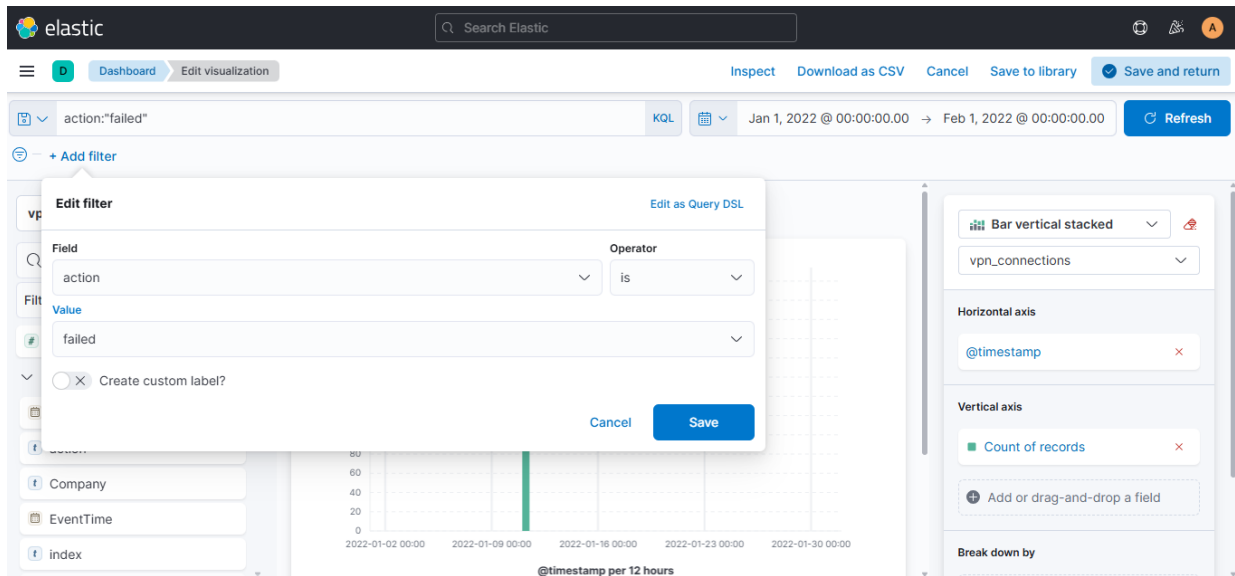


Figure 30. Attaching action: failed as a panel-level filter instead of a dashboard-wide query.

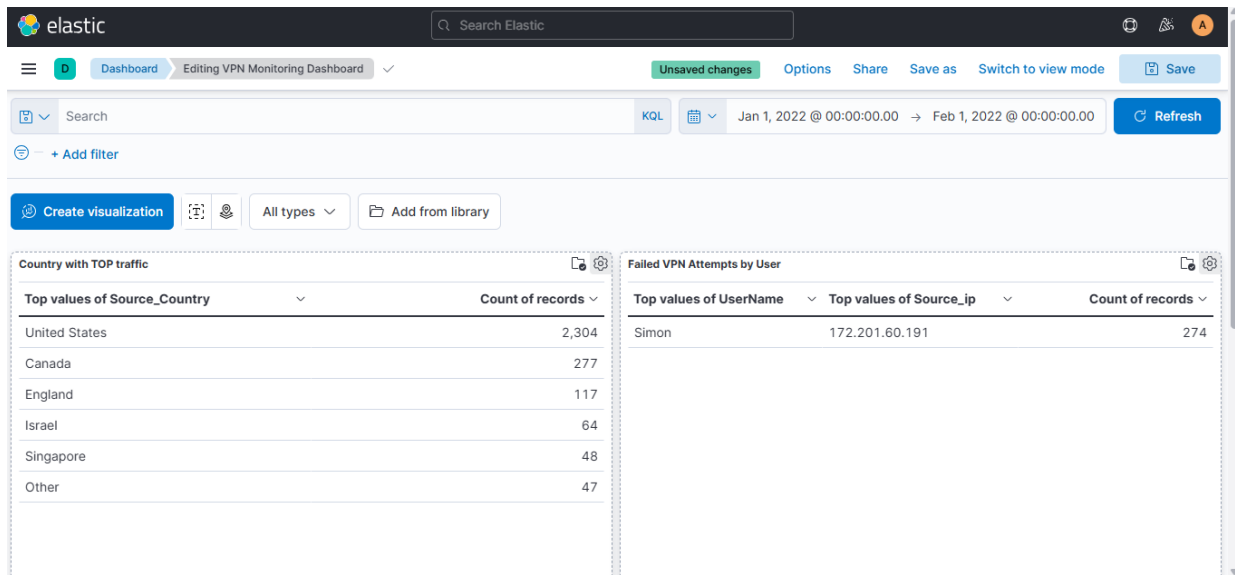


Figure 31. Search bar cleared — the monitoring panels show all traffic while the timeline panel keeps its own filter.

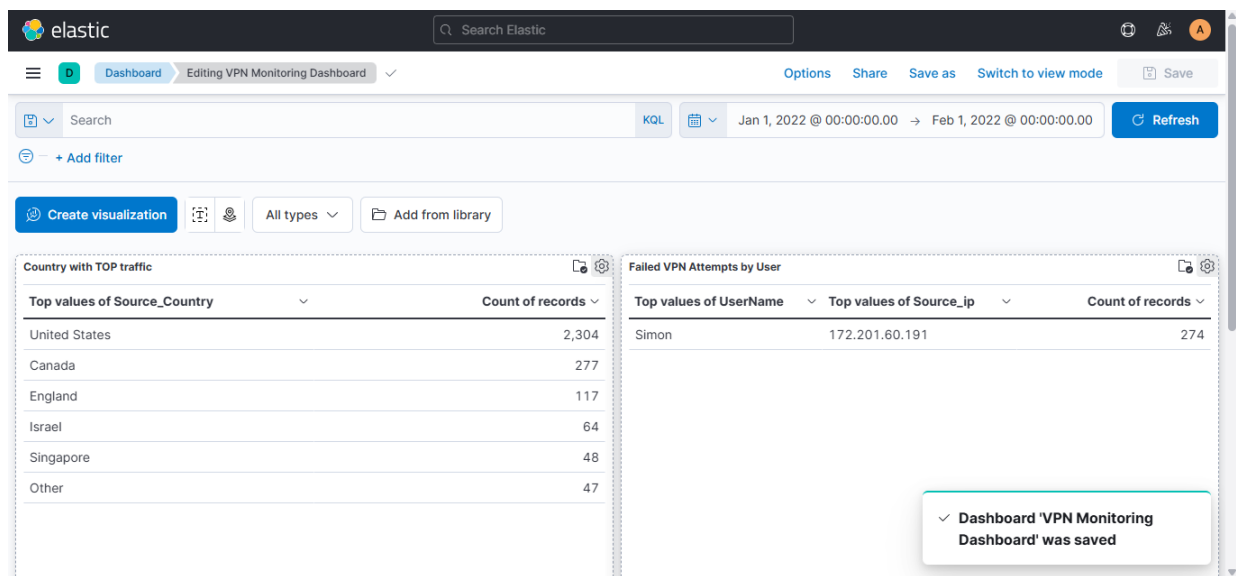


Figure 32. Dashboard saved with all three panels verified.

Step 8 — Pivoted to the question that matters: did they get in?

Failed logins only tell half the story. I flipped the query to successful connections from the attacker's IP — `Source_ip:"172.201.60.191"` and `action:"built"` — and got three hits, all on Simon's account. The first was January 11 at 03:35:27, about two hours after the failed burst ended. The second and third were January 12 and January 13, both at exactly 04:35:27 — twenty-four hours apart to the second. People are not that punctual. Scripts are. This was not just a compromised account; it was automated, scheduled re-entry.

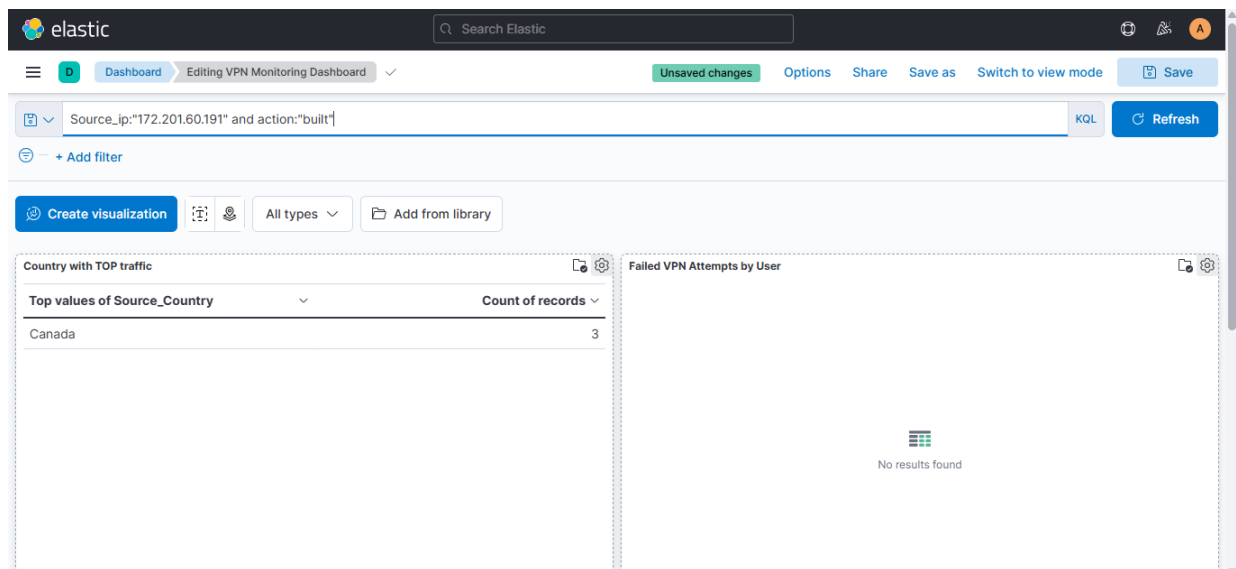


Figure 33. Pivoting on the dashboard: successful connections from the attacker IP — 3 events, all from Canada.

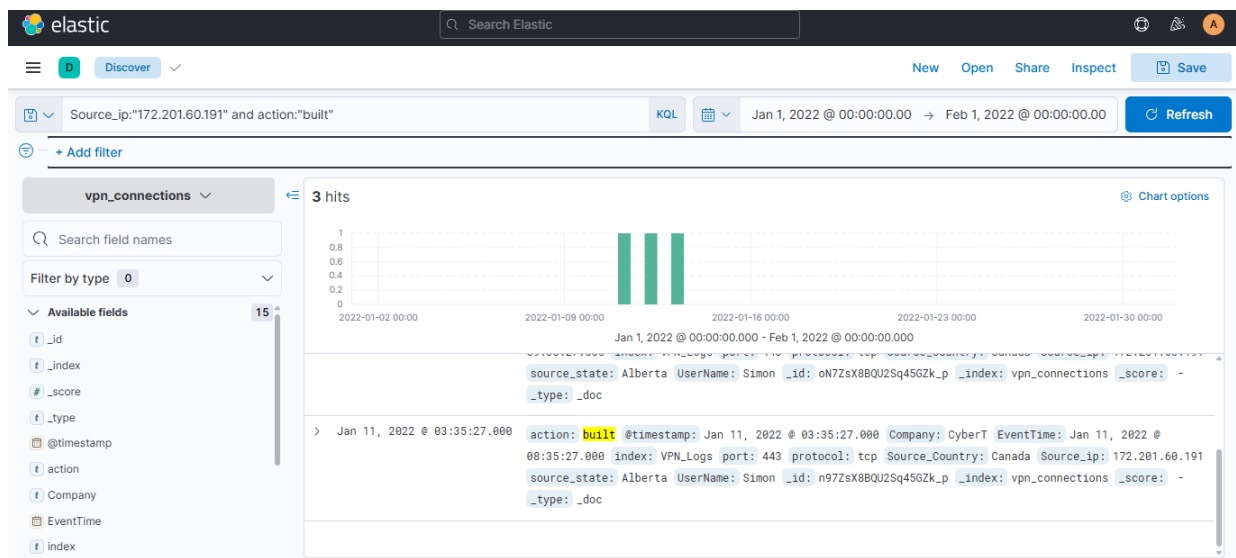


Figure 34. Discover confirms the first successful login: Jan 11 at 03:35:27, roughly two hours after the brute-force burst.

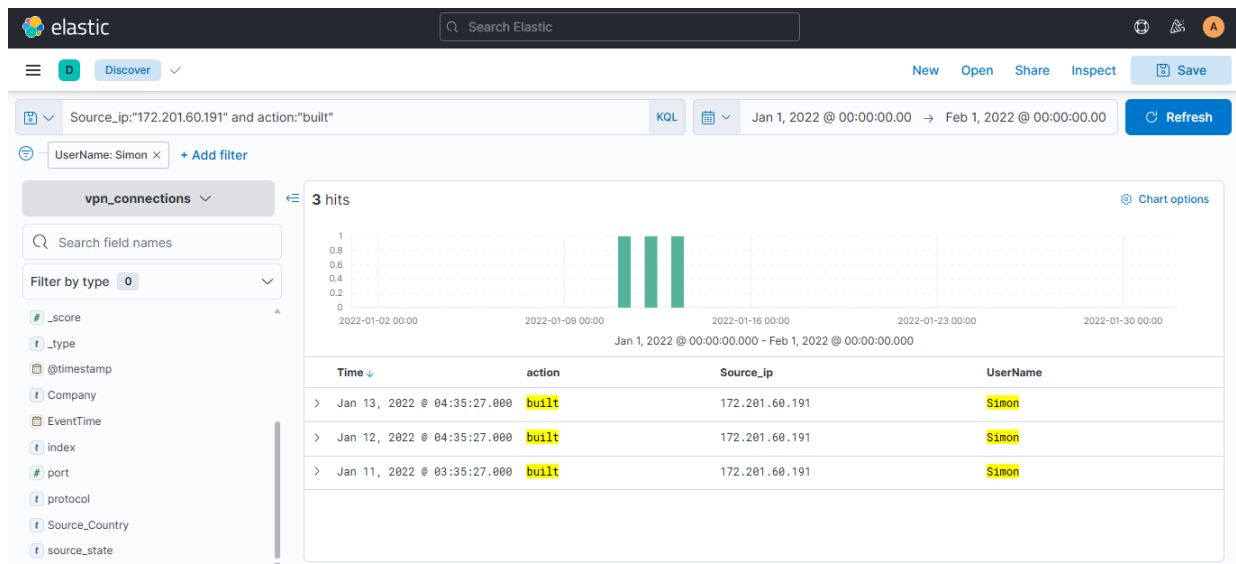


Figure 35. All three successful logins on Simon's account — the last two exactly 24 hours apart, to the second.

Investigation Summary

The investigation started with basics that are easy to skip: get the time window right, and understand the shape of the data before querying it. Field statistics in Discover gave me the top talkers and most active users in seconds, which set a baseline for what "normal" looked like in this environment. That baseline is what made the anomalies stand out later.

Two findings came out of the guided tasks that would each justify a ticket on their own. First, a terminated employee's account successfully connected to the VPN six days after termination — that is an offboarding process failure, and in a real environment I would flag it for identity and HR follow-up regardless of whether it turned out to be malicious. Second, one user, Simon, accounted for every single failed VPN login in the entire month, all from one IP address in Alberta, Canada.

The extension work was about turning that second finding from a count into a story. Discovering the action field's three values (built, failed, teardown) gave me the vocabulary to slice the data properly. The timeline chart showed the 274 failures were not spread across the month — they were compressed into a single overnight hour on January 11 at four to five attempts per minute, which is machine speed, not human speed. Pivoting from failed to successful connections closed the loop: the same IP logged in as Simon two hours after the burst, then again the next two nights at exactly the same second each time. My read is that the brute-force either succeeded or the attacker obtained the password another way shortly after, and the clockwork 24-hour re-entries are a script maintaining access.

Two decisions during the analysis were about data quality rather than the attack itself, and I think they matter just as much. When my first timeline chart rendered, I checked the record count before trusting it — it said 2,857, which meant my filter had not applied and the chart was wrong. And when I added the chart to the dashboard, I deliberately scoped the failed filter to the panel rather than the dashboard search bar, so the monitoring panels keep showing all traffic while the timeline stays accurate. Getting the numbers right before drawing conclusions is the job.

If this were a live SOC environment, my escalation would be: disable or lock Simon's account and force a credential reset, block 172[.]201[.]60[.]191 at the VPN gateway, review what Simon's account touched after each of the three successful logins, verify MFA enforcement on the VPN, and open a separate ticket for the offboarding gap that left Johny Brown's account active after termination.

Results / Findings

- Confirmed brute-force attack: 274 failed VPN logins against the account Simon, all from 172[.]201[.]60[.]191 (Alberta, Canada), all inside a single overnight hour on January 11, 2022 — roughly 4–5 attempts per minute.
- Confirmed account compromise: the same IP successfully authenticated as Simon on January 11 at 03:35:27, about two hours after the failed burst.
- Automated persistence: further successful logins on January 12 and January 13, both at exactly 04:35:27 — 24 hours apart to the second, consistent with a scheduled script rather than a person.
- Offboarding gap: terminated employee Johny Brown's account built a successful VPN connection on January 7 at 03:28:47 from 175[.]20[.]60[.]191, six days after his January 1 termination.
- Environment baseline: 2,857 VPN events in January 2022; most traffic from the United States (2,304 events); busiest source IP 238[.]163[.]231[.]224; most active user James.
- Deliverable: a saved, working VPN Monitoring Dashboard with three panels — traffic by country, failed attempts by user, and a correctly filtered failed-attempts timeline.

Indicators of interest (defanged):

Indicator	Type	Context
172[.]201[.]60[.]191	IP address (Alberta, Canada)	Source of all 274 failed VPN logins and 3 successful logins on the compromised account

Simon	User account	Targeted and compromised account; only user with failed logins in January
175[.]20[.]60[.]191	IP address	Source of a successful VPN connection by a terminated employee's account (Jan 7)
238[.]163[.]231[.]224	IP address	Highest-volume source IP in the dataset; flagged for follow-up review

Skills Demonstrated

- SIEM log analysis and threat hunting with the Elastic Stack (Kibana Discover, Lens, Dashboards)
- KQL query development — free text, wildcards, field:value matching, and/or/not boolean logic
- Alert triage and investigation methodology, from baseline profiling to root cause
- Brute-force attack detection, account compromise confirmation, and persistence identification
- Attack timeline reconstruction using timestamp analysis
- Security dashboard development for continuous monitoring, including filter scoping (panel-level vs dashboard-level)
- Data validation during analysis — verifying record counts before trusting visualizations
- Identification of process gaps (user offboarding / access revocation failure)
- Security documentation and reporting with defanged indicators of compromise

What I Learned

The biggest lesson from this lab was that a number is not a finding until you know its shape. "274 failed logins" sounds bad, but it only became meaningful when I saw all 274 packed into one hour — that is what separates a user forgetting their password from an automated attack. I also learned to distrust my own charts until the record count agrees with what I expect; my first timeline was silently counting everything, and I would have drawn wrong conclusions from a nice-looking graph.

Practically, I got comfortable moving between Discover, Lens and Dashboards without losing my filters, and I now understand the difference between a dashboard-wide query and a panel-level filter — I learned that the hard way when my search broke both panels. The pivot from failed to successful logins is the habit I most want to keep: the failures got my attention, but the three quiet successful logins were the actual incident. And timestamps carry more information than I gave them credit for — two logins exactly 24 hours apart, to the second, told me more about the attacker than the 274 failures did.